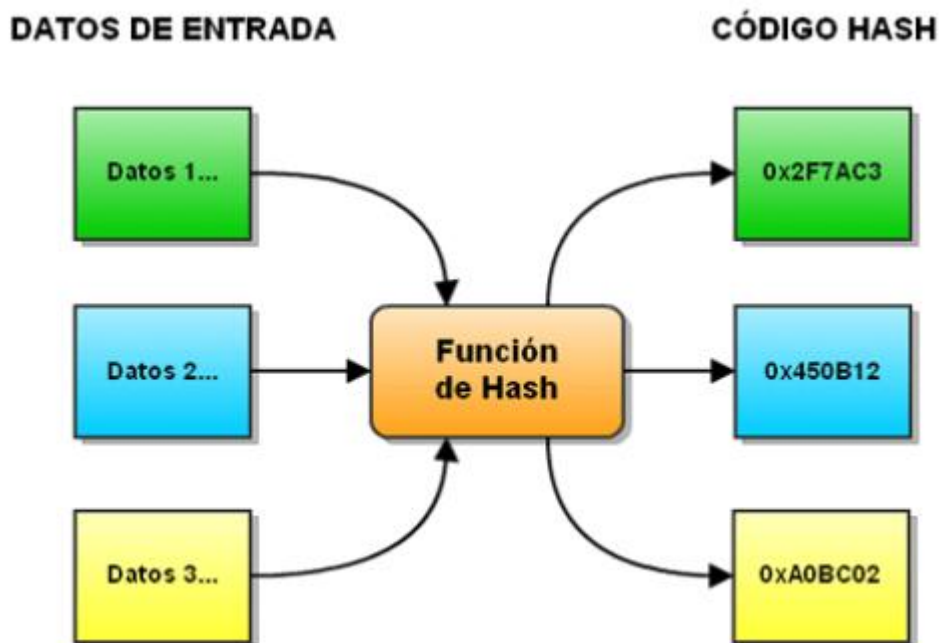


Realizar una investigación sobre la búsqueda por llaves también conocida como hashing, esta investigación deberá incluir:

Definición de búsqueda por llave o hashing

Hash se refiere a una función o método para generar claves o llaves que representen de manera casi unívoca a un registro, archivo, documentó, etc.

Una función de hash es una función para resumir o identificar probabilísticamente un gran conjunto de información, dando como resultado un conjunto imagen finito generalmente menor (un subconjunto de los números naturales, por ejemplo).



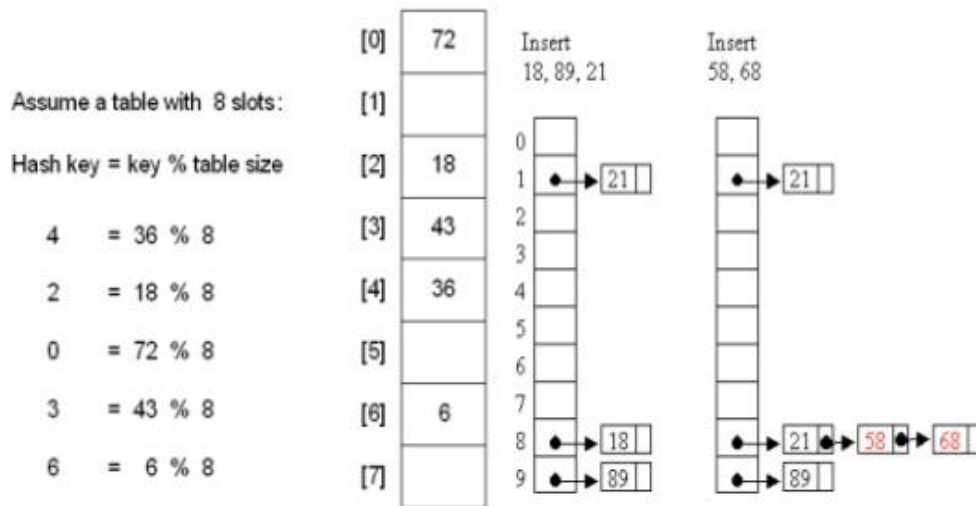
¿Qué es un algoritmo hash y para qué se utiliza?

Es un método de búsqueda que aumenta la velocidad de búsqueda, pero que no requiere que los elementos estén ordenados. Consiste en asignar a cada elemento un índice mediante una transformación del elemento. Esta correspondencia se realiza mediante una función de conversión, llamada función hash. La correspondencia más sencilla es la identidad, esto es, al número 0 se le asigna el índice 0, al elemento 1 el índice 1, y así sucesivamente.

Pero si los números a almacenar son demasiado grandes esta función es inservible. Por ejemplo, se quiere guardar en un array la información de los 1000 usuarios de una empresa, y se elige el número de DNI como elemento identificativo. Es inviable hacer un array de 100.000.000

elementos, sobre todo porque se desaprovecha demasiado espacio. Por eso, se realiza una transformación al número de DNI para que nos de un número menor, por ejemplo coger las 3 últimas cifras para guardar a los empleados en un array de 1000 elementos. Para buscar a uno de ellos, bastaría con realizar la transformación a su DNI y ver si está o no en el array.

La función de hash ideal debería ser biyectiva, esto es, que a cada elemento le corresponda un índice, y que a cada índice le corresponda un elemento, pero no siempre es fácil encontrar esa función, e incluso a veces es inútil, ya que puedes no saber el número de elementos a almacenar. La función de hash depende de cada problema y de cada finalidad.



¿Qué es una tabla hash?

Es una estructura de datos no lineal cuyo propósito final se centra en llevar a cabo las acciones básicas (inserción, eliminación y búsqueda de elementos) en el menor tiempo posible, mejorando las cotas de rendimiento respecto a un gran número de estructuras.

Una tabla hash está formada por un array de entradas, que será la estructura que almacene la información, y por una función de dispersión. La función de dispersión permite asociar el elemento almacenado en una entrada con la clave de dicha entrada. Por lo tanto, es un algoritmo crítico para el buen funcionamiento de la estructura.

Existen dos tipos de tablas hash, en función de cómo resuelven las colisiones:

- Encadenamiento separado: Las colisiones se resuelven insertándolas en una lista. De esa forma tendríamos como estructura un vector de listas. Al número medio de claves por lista se le llama *factor de carga* y habría que intentar que esté próximo a 1.
- Direccinamiento abierto: Utilizamos un vector como representación y cuando se produzca una colisión la resolvemos reasignándole otro valor hash a la clave hasta que encontremos un hueco.

¿Qué se entiende por Colisión?

Un problema potencial encontrado en este proceso, es que tal función no puede ser uno a uno; las direcciones calculadas pueden no ser todas únicas, cuando $R(k_1) = R(k_2)$ Pero k_1 diferente de k_2 decimos que hay una colisión.

Cuando dos claves se direccionan a la misma dirección o bucket se dice que hay una colisión, y a las claves se les denomina sinónimos.

Cuando hay colisiones se requiere de un proceso adicional para encontrar una posición disponible para la clave. Esto obviamente degrada la eficiencia del método, por lo que se trata de evitar al máximo esta situación. Una función de hashing que logra evitar al 100% las colisiones son conocida como hashing perfecto.

Ahora bien, cuando se tienen dos números que generan la misma dirección se tiene una colisión. Por lo tanto, se necesita solucionar la colisión.

Al menos un método de resolver las colisiones

- Direccionamiento abierto

Cuando el número n de elementos en la tabla se puede estimar con anticipación, existen métodos para almacenar n registros en una tabla de tamaño m donde $m > n$, los cuales utilizan entradas vacías de la tabla para resolver colisiones, esos métodos se denominan métodos de direccionamiento abierto. Cuando una llave x se direcciona a una entrada de la tabla que ya está ocupada, se elige una secuencia de localizaciones alternativas $h_1(x)$, $h_2(x)$... dentro de la tabla. Si ninguna de las $h_1(x)$, $h_2(x)$... posiciones se encuentra vacía, entonces la tabla está llena y x no se puede insertar.

- Enlazamiento

Consiste en colocar los elementos mapeados a la misma dirección de la tabla hash en una lista ligada. La posición o dirección j contiene un apuntador al inicio de la una lista ligada que contiene todos los elementos que son mapeados a la posición j . Si no hay elementos entonces la localidad tendrá un null.

- Reasignación.

Prueba lineal. Consiste en una vez que se detecta una colisión, recorrer el arreglo secuencialmente a partir del punto de colisión hasta que se encuentra un lugar disponible para el elemento. En caso de que se esté buscando, el elemento se deberá recorrer el arreglo a partir de la posición en donde se produjo la colisión hasta que se encuentre al elemento o se llegue nuevamente a la posición indicada considerando que se tiene un arreglo circular.

- Arreglos anidados

Este método en que cada elemento del arreglo tenga otro arreglo, en el cuál, se almacenen los elementos colisionados.

- Encadenamiento

Consiste en que cada elemento del arreglo tenga un apuntador a una lista ligada, la cual se irá generando e irá almacenando los valores colisionados a medida que se requiera.

Método Progressive Overflow

El algoritmo es el más popular y de hecho bastante simple:

- Cuando se inserta un registro, se obtiene su dirección con la función Hash.
- Si la dirección esta libre entonces se inserta ahí.
- En caso contrario se busca la primera dirección subsecuente que esté disponible y ahí

se inserta.

- Es importante recordar que el archivo puede tener un número fijo de direcciones posibles, de manera que si se alcanza ese número máximo de direcciones entonces se busca buscar un espacio disponible desde el inicio del archivo. Si no queda ninguno entonces quizás hubo un error en los cálculos y planeación del archivo.

Para las búsquedas, se obtiene el hash correspondiente.

A parte de esa dirección se comienza a buscar por el registro deseado o un espacio vacío. En caso de encontrar un espacio totalmente vacío (nuevo, no reutilizable), entonces, se afirma que el registro no se encuentra en el archivo. Si se alcanza el final del archivo y no se encuentra el registro buscado ni un espacio vacío entonces se sigue buscando desde el inicio del archivo hasta volver al punto de partida (dirección original del hash).

Bibliografía

<https://edukativos.com/apuntes/archives/10585>

http://artemisa.unicauca.edu.co/~nediaz/EDDI/cap02.htm#ancla2_4

<https://es.slideshare.net/Tonkseverus1/mtodo-de-bsqueda-hash>

<https://ccia.ugr.es/~jfv/ed1/tedi/cdrom/docs/tablash.html>